evangelionPL

Manual de usuario y manual técnico. Fases 1, 2 y 3

Juan Pablo Orjuela Rojas
Lenguajes de Programación y Transducción
Ciencias de la Computación e Inteligencia Artificial
Universidad Sergio Arboleda, 2026-2

1. Qué es evangelionPL

evangelionPL es un lenguaje de dominio específico para trabajar con tablas de datos: cargar un CSV, escoger columnas, filtrar filas, crear columnas calculadas, agrupar, sacar estadísticas y guardar el resultado. Es lo que pide el proyecto del curso, solo que las instrucciones se llaman como cosas de Neon Genesis Evangelion. Los datos entran por el `ENTRY_PLUG`, el `AT_FIELD` deja pasar únicamente las filas que cumplen una condición y las `MAGI` hacen los cálculos.

La idea central es el flujo: se parte de una tabla y se le encadenan etapas con `|>`. Cada etapa recibe una tabla y entrega otra nueva, así que el programa se lee de arriba hacia abajo como una lista de pasos. El lenguaje no pretende ser otro Python: solo sabe hacer estas operaciones, pero las dice corto.

Está hecho con ANTLR4 y Python. Para manejar los datos no se usa pandas ni ninguna otra librería: todo lo hace una librería propia de tablas, escrita para este proyecto, y las gráficas tampoco usan Matplotlib: se dibujan píxel por píxel y el archivo PNG se arma desde cero. Este documento cubre las tres fases y sirve como manual de usuario (secciones 1, 2, 9 y 10) y como manual técnico (3 a 8).

Fase	Qué pide el curso	Estado
1	Especificación del lenguaje, gramática, lexer y parser, árbol de análisis y errores léxicos y sintácticos.	Completa
2	Visitor, tabla de símbolos, ejecución sobre datos, agregaciones, exportar a CSV y errores semánticos.	Completa
3	Gráficas con títulos, ejes, leyendas y archivos de salida, ejecución desde consola, caso de estudio con datos reales y pruebas de regresión.	Completa

Un primer programa

```
# Mi primer programa: cargar las pruebas y ver las cinco mejores
NERV "Primera prueba de sincronizacion"

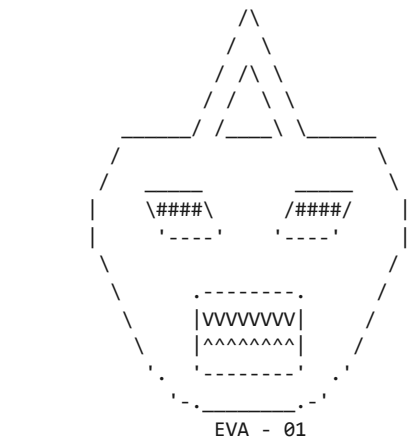
pruebas = ENTRY_PLUG "datos/pruebas_sincronizacion.csv"

mejores = pruebas
    |> TARGET [fecha, piloto, unidad, sincronizacion]
    |> AT_FIELD donde sincronizacion >= 60 y piloto != "Kaworu"
    |> FORMATION por [sincronizacion desc]
    |> BATTERY 5

MONITOR mejores
```

Al ejecutarlo con `python run.py ejemplos/01_primera_prueba.eva` sale la tabla y, como todo estuvo bien, el EVA-01 felicita:

```
MONITOR mejores (5 filas x 4 columnas)
fecha       piloto  unidad  sincronizacion
2015-10-19  Shinji  EVA-01           93.50
2015-07-20  Asuka   EVA-02           92.70
2015-08-03  Asuka   EVA-02           86.60
2015-10-05  Shinji  EVA-01           80.10
2015-09-07  Rei     EVA-00           77.90
```




```
¡Felicidades, Shinji! (omedetou!)
Sincronización con el EVA-01 al 100 %: 01_primera_prueba.eva corrió sin errores.

  Operación : Primera prueba de sincronizacion
  Tokens    : 42
  Variables : pruebas, mejores
```

2. Catálogo de instrucciones

Las instrucciones van en mayúscula y son términos de la serie. Los conectores (`donde`, `por`, `como`, `con`, `de`) van en minúscula y en español. La tercera columna cuenta por qué cada instrucción se llama así.

Instrucción	Qué hace	Por qué ese nombre
`NERV "título"`	Le pone nombre a la operación. Es opcional y va en la primera línea.	NERV es la organización que dirige todo.
`ENTRY_PLUG "archivo.csv"`	Carga un archivo CSV como tabla.	Es la cápsula por donde el piloto entra al EVA; aquí es por donde entran los datos.
`TARGET [a, b]`	Deja solo las columnas de la lista.	Fijar el blanco antes de disparar.
`AT_FIELD donde condición`	Deja pasar solo las filas que cumplen la condición.	El campo AT es la barrera que no deja pasar casi nada.
`SYNC col = expresión`	Crea una columna calculada (o varias, separadas por coma).	La sincronización entre piloto y EVA.
`CODENAME [vieja -> nueva]`	Les cambia el nombre a las columnas.	Los nombres clave.
`FORMATION por [col desc]`	Ordena las filas; cada columna puede ir asc o desc.	La formación en la que salen las unidades.

Instrucción	Qué hace	Por qué ese nombre
`EJECT duplicados | faltantes | columnas`	Saca filas repetidas, filas con vacíos o columnas completas.	La eyección de la cápsula.
`DUMMY_PLUG col = valor`	Rellena los vacíos de una columna.	El dummy plug reemplaza al piloto cuando falta.
`EQUIP col -> tipo`	Convierte la columna a entero, real, texto o booleano.	Los equipos (tipo B, tipo D) que le cambian al EVA.
`SQUAD por [a, b]`	Arma los grupos. Después siempre va MAGI.	El escuadrón.
`MAGI n = contar(), m = media(col)`	Calcula un resumen por grupo, o de toda la tabla si no hay grupos.	Las supercomputadoras que hacen todos los cálculos de NERV.
`BATTERY n`	Deja las primeras n filas.	La batería interna, que dura poquito.
`DIAGNOSTIC`	Estadísticas de cada columna: válidos, faltantes, distintos, media, desviación, mínimo y máximo.	El chequeo de la unidad.
`MONITOR flujo`	Muestra una tabla o un valor en la terminal.	Las pantallas del centro de mando.
`SEELE flujo como "archivo.csv"`	Guarda una tabla en un CSV.	El comité al que NERV le entrega informes.
`PATTERN_BLUE cond { } PATTERN_ORANGE { }`	Condicional: ejecuta el primer bloque si se cumple y, si no, el segundo.	Patrón azul es ángel confirmado; patrón naranja, sin identificar.
`HOLOGRAM tipo de tabla { }`	Dibuja una gráfica (barras, lineas, histograma, dispersion o caja) y la guarda en PNG o SVG.	Los hologramas de las reuniones de SEELE.

Funciones

Función	Uso
`suma media mediana desviacion`	Agregaciones sobre una columna numérica.
`minimo maximo contar`	Agregaciones sobre cualquier columna. `contar()` sin columna cuenta filas.
`filas(tabla)`	Cuántas filas tiene una tabla.
`redondear(x, n)`, `absoluto(x)`	Para números o columnas numéricas.
`mayusculas(t)`, `minusculas(t)`	Para textos o columnas de texto.
`reemplazar(t, viejo, nuevo)`	Cambia un pedazo de texto por otro, por ejemplo para traducir los nombres de los países.

Los tipos de `EQUIP` son entero, real, texto y booleano. Los tipos de `HOLOGRAM` son barras, lineas, histograma, dispersion y caja. Ninguno es palabra reservada: se validan en la fase semántica, para que una columna pueda

llamarse, por ejemplo, `texto`.

3. Fase 1: análisis léxico

Toda la gramática está en un solo archivo, `grammar/Eva.g4`: arriba las reglas sintácticas y abajo las léxicas. De ahí ANTLR saca el lexer, que lee el archivo carácter por carácter y entrega tokens. Estas tablas se generan a partir del lexer que produce ANTLR, así que siempre coinciden con la gramática.

Categoría	Tokens (nombre y lexema)
Instrucciones	`NERV`, `ENTRY_PLUG`, `MONITOR`, `SEELE`, `HOLOGRAM`, `PATTERN_BLUE`, `PATTERN_ORANGE`
Etapas del flujo	`TARGET`, `AT_FIELD`, `SYNC`, `CODENAME`, `FORMATION`, `EJECT`, `DUMMY_PLUG`, `EQUIP`, `SQUAD`, `MAGI`, `BATTERY`, `DIAGNOSTIC`
Conectores	`DONDE donde`, `POR por`, `COMO como`, `CON con`, `DE de`, `ASC asc`, `DESC desc`, `DUPLICADOS duplicados`, `FALTANTES faltantes`, `COLUMNAS columnas`
Lógicos y literales fijos	`Y y`, `O o`, `NO no`, `VERDADERO verdadero`, `FALSO falso`, `NULO nulo`
Operadores y signos	`TUBERIA |>`, `FLECHA ->`, `IGUAL_IGUAL ==`, `DIFERENTE !=`, `MENOR_IGUAL <=`, `MAYOR_IGUAL >=`, `MENOR <`, `MAYOR >`, `IGUAL =`, `MAS +`, `MENOS -`, `POR_OP *`, `DIV /`, `MODULO %`, `POTENCIA ^`, `PAR_IZQ (`, `PAR_DER )`, `COR_IZQ [`, `COR_DER ]`, `LLAVE_IZQ {`, `LLAVE_DER }`, `COMA ,`, `DOS_PUNTOS :`, `PUNTO .`

Token	Qué reconoce
`REAL`	dígitos, un punto y más dígitos: 0.75
`ENTERO`	uno o más dígitos: 42
`CADENA`	texto entre comillas dobles: "Tokio-3"
`CADENA_SIN_CERRAR`	comilla que nunca se cierra; se reporta como error léxico
`ID`	letra o guion bajo seguido de letras, dígitos o guiones bajos: sync_01
`ID_ESCAPADO`	cualquier texto entre acentos graves: `` `Unidad EVA` ``
`COMENTARIO`	desde # hasta el final de la línea; se descarta
`ESPACIOS`	espacios, tabulaciones y saltos de línea; se descartan
`CARACTER_INVALIDO`	cualquier otro carácter; se reporta como error léxico

Decisiones del lexer

Las palabras reservadas se declaran antes que `ID`. Cuando dos reglas reconocen el mismo texto, ANTLR se queda con la primera, y por eso `MAGI` sale como instrucción y no como nombre. Si el texto es más largo (`MAGI_2`) gana `ID`, porque siempre se prefiere el lexema más largo. Las reservadas distinguen mayúsculas: `magi` es un nombre cualquiera.

`REAL` va antes que `ENTERO` para que `3.5` no se parta en tres tokens. Los comentarios y los espacios se descartan con `-> skip` y el parser nunca los ve.

Las palabras `y`, `o` y `no` son operadores, así que no sirven como nombres de variable. Para una columna que se llame así, o que tenga espacios, se usan acentos graves: `` `Unidad EVA` ``.

Hay dos tokens que existen solo para atrapar errores: `CARACTER_INVALIDO` y `CADENA_SIN_CERRAR`. Con ellos el lexer nunca se queda atascado y el error se reporta con la línea y la columna exactas.

Los tokens de un programa

Con `--tokens` se ve lo que entrega el lexer. Estos son los primeros del ejemplo anterior:

```
TOKENS de 01_primera_prueba.eva
   #  LIN:COL  TIPO               LEXEMA
   1  2:1      NERV               NERV
   2  2:6      CADENA             "Primera prueba de sincronizacion"
   3  4:1      ID                 pruebas
   4  4:9      IGUAL              =
   5  4:11     ENTRY_PLUG         ENTRY_PLUG
   6  4:22     CADENA             "datos/pruebas_sincronizacion.csv"
   7  6:1      ID                 mejores
   8  6:9      IGUAL              =
   9  6:11     ID                 pruebas
  10  7:5      TUBERIA            |>
  11  7:8      TARGET             TARGET
  12  7:15     COR_IZQ            [
  13  7:16     ID                 fecha
  14  7:21     COMA               ,
 ...
```

4. Fase 1: análisis sintáctico

Las reglas sintácticas son la primera parte de `grammar/Eva.g4` y de ellas sale el parser. Esta es la gramática en EBNF:

```
(* Gramatica de evangelionPL en EBNF.
   { x } = cero o mas veces, [ x ] = opcional, | = alternativa *)

programa       = [ cabecera ] , { sentencia } ;
cabecera       = "NERV" , CADENA ;

sentencia      = asignacion | mostrar | guardar | graficar | condicional ;
asignacion     = nombre , "=" , flujo ;
mostrar        = "MONITOR" , flujo ;
guardar        = "SEELE" , flujo , "como" , CADENA ;
graficar       = "HOLOGRAM" , ID , "de" , flujo , [ "{" , { opcion } , "}" ] ;
condicional    = "PATTERN_BLUE" , expresion , bloque , [ "PATTERN_ORANGE" , bloque ] ;
bloque         = "{" , { sentencia } , "}" ;

flujo          = fuente , { "|>" , etapa } ;
fuente         = carga | expresion ;
carga          = "ENTRY_PLUG" , CADENA , [ "con" , "{" , { opcion } , "}" ] ;
opcion         = ID , ":" , expresion , [ "," ] ;

etapa          = "TARGET" , lista_columnas
               | "AT_FIELD" , "donde" , expresion
               | "SYNC" , columna_nueva , { "," , columna_nueva }
               | "CODENAME" , "[" , renombre , { "," , renombre } , "]"
               | "FORMATION" , "por" , "[" , criterio , { "," , criterio } , "]"
               | "EJECT" , "duplicados" , [ lista_columnas ]
               | "EJECT" , "faltantes" , [ lista_columnas ]
               | "EJECT" , "columnas" , lista_columnas
               | "DUMMY_PLUG" , columna_nueva , { "," , columna_nueva }
               | "EQUIP" , nombre , "->" , ID
               | "SQUAD" , "por" , lista_columnas
```

```
                | "MAGI" , agregacion , { "," , agregacion }
                | "BATTERY" , ENTERO
                | "DIAGNOSTIC" ;

  columna_nueva  = nombre , "=" , expresion ;
  agregacion     = nombre , "=" , ID , "(" , [ nombre ] , ")" ;
  renombre       = nombre , "->" , nombre ;
  criterio       = nombre , [ "asc" | "desc" ] ;
  lista_columnas = "[" , nombre , { "," , nombre } , "]" ;

  (* Expresiones: cada nivel amarra mas fuerte que el anterior *)
  expresion      = disyuncion ;
  disyuncion     = conjuncion , { "o" , conjuncion } ;
  conjuncion     = negacion , { "y" , negacion } ;
  negacion       = "no" , negacion | igualdad ;
  igualdad       = comparacion , { ( "==" | "!=" ) , comparacion } ;
  comparacion    = suma , { ( "<" | "<=" | ">" | ">=" ) , suma } ;
  suma           = termino , { ( "+" | "-" ) , termino } ;
  termino        = unario , { ( "*" | "/" | "%" ) , unario } ;
  unario         = "-" , unario | potencia ;
  potencia       = primario , [ "^" , unario ] ;
  primario       = literal
                 | nombre
                 | nombre , "." , nombre
                 | ID , "(" , [ expresion , { "," , expresion } ] , ")"
                 | "(" , expresion , ")" ;

  literal        = ENTERO | REAL | CADENA | "verdadero" | "falso" | "nulo" ;
  nombre         = ID | ID_ESCAPADO ;

  (* Lexico *)
  ENTERO         = digito , { digito } ;
  REAL           = digito , { digito } , "." , digito , { digito } ;
  CADENA         = '"' , { caracter } , '"' ;
  ID             = letra , { letra | digito | "_" } ;
  ID_ESCAPADO    = "`" , caracter , { caracter } , "`" ;
  comentario     = "#" , { caracter } ;
```

Precedencia de los operadores

Nivel	Operadores	Asociatividad
1 (más fuerte)	`( )`, llamadas, `tabla.columna`	—
2	`^`	derecha
3	`-` de signo	—
4	`* / %`	izquierda
5	`+ -`	izquierda
6	`< <= > >=`	izquierda
7	`== !=`	izquierda
8	`no`	—
9	`y`	izquierda
10 (más débil)	`o`	izquierda

En la EBNF la precedencia se ve por niveles (suma, término, potencia…). En el archivo `.g4` es una sola regla `expresion` recursiva por la izquierda, y ANTLR saca la precedencia del orden de las alternativas: la que está más arriba amarra más fuerte. Las dos formas describen lo mismo, con

una diferencia pequeña: el `.g4` también acepta un `no` como operando (`a == no b`). La potencia es asociativa a la derecha, así que `2 ^ 3 ^ 2` es 2 ^ 9, y `-2 ^ 2` da -4 como en matemáticas.

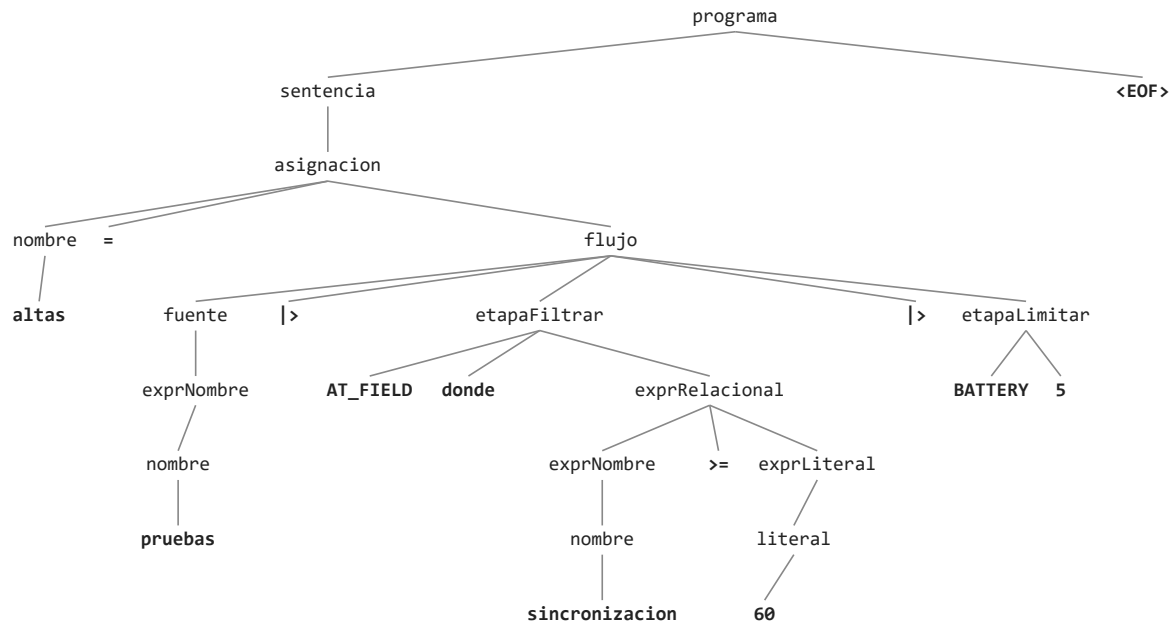
No hay punto y coma. Cada sentencia empieza con una instrucción o con `nombre =`, y con eso el parser sabe dónde termina la anterior. La regla `programa` termina en `EOF` para que no se pueda colar basura al final del archivo.

Árbol de análisis

Para esta línea:

```
altas = pruebas |> AT_FIELD donde sincronizacion >= 60 |> BATTERY 5
```

el parser arma el siguiente árbol. Se ve que la comparación quedó colgando de la etapa `AT_FIELD` y que las dos etapas son hermanas dentro del mismo flujo.



Árbol generado con `python run.py archivo.eva --svg arbol.svg`

El mismo árbol en texto, con `--arbol`:

```
programa
|-- sentencia
|   `-- asignacion
|       |-- nombre
|       |   `-- altas
|       |-- =
|       `-- flujo
|           |-- fuente
|           |   `-- exprNombre
|           |       `-- nombre
|           |           `-- pruebas
|           |-- |>
|           |-- etapaFiltrar
|           |   |-- AT_FIELD
|           |   |-- donde
|           |   `-- exprRelacional
|           |       |-- exprNombre
|           |       |   `-- nombre
|           |       |       `-- sincronizacion
|           |       |-- >=
|           |       `-- exprLiteral
|           |           `-- literal
|           |               `-- 60
|           |-- |>
|           `-- etapaLimitar
|               |-- BATTERY
|               `-- 5
`-- <EOF>
```

Los nodos llevan el nombre de la alternativa (`etapaFiltrar`, `exprRelacional`) y no solo el de la regla. Esas etiquetas son las que después usa el Visitor: ANTLR genera un método `visit` por cada una.

5. Fase 2: semántica y ejecución

Si el programa pasa la fase 1, el árbol se recorre con el patrón Visitor (`src/evangelion/interprete.py`). La clase `Interprete` hereda del `EvaVisitor` que genera ANTLR y tiene un método por cada tipo de nodo: `visitEtapaFiltrar`, `visitExprAditiva`, `visitGuardar`... Cada método revisa que lo que le llegó tenga sentido y después le pide el trabajo a la librería de tablas.

```
programa .eva → lexer → tokens → parser → árbol → Visitor (revisa y ejecuta) → librería de tablas →
                                                                                tablas y CSV
```

Tabla de símbolos

Cada asignación guarda en la tabla de símbolos el nombre, la clase del valor, el valor y la línea donde se definió. Hay un solo ámbito global: lo que se asigna dentro de un `PATTERN_BLUE` sigue existiendo afuera. Con `--simbolos` se imprime al terminar; así queda después del ejemplo 04:

```
TABLA DE SÍMBOLOS
  NOMBRE             CLASE     LÍNEA  VALOR
  pruebas            tabla     4      29 filas x 7 columnas
  asuka              tabla     8      8 filas x 4 columnas
  ultimas            tabla     17     3 filas x 4 columnas
  promedio_total     numero    19     66.6
  promedio_reciente  numero    20     47.9
  alerta             tabla     25     3 filas x 5 columnas
```

Las clases de valor son tabla, numero, texto, booleano y nulo. Una columna suelta (`tabla.columna`) se puede usar dentro de una expresión, pero no guardar en una variable.

Reglas semánticas

Sobre	Regla
Variables	Una variable tiene que estar asignada antes de usarse. Se puede volver a asignar.
Columnas	Toda columna que se nombre en una etapa tiene que existir en la tabla que llega. Si no existe, el error sugiere la que más se parece.
Nombres dentro de una etapa	Primero se busca entre las columnas de la tabla y después entre las variables.
Aritmética	+ - * / % ^ solo aceptan números. El + también pega dos textos. Dividir un valor por cero es un error; dentro de una columna, esa fila queda como faltante.
Comparaciones	< <= > >= comparan número con número o texto con texto. == y != piden la misma clase a los dos lados, salvo que uno sea nulo.
Lógica	y, o, no solo aceptan valores de verdadero/falso.
Condiciones	La condición de AT_FIELD y la de PATTERN_BLUE tienen que dar verdadero o falso.
Etapas	Una etapa solo se aplica a una tabla. Ninguna cambia la tabla original: devuelve una nueva.
Grupos	Después de SQUAD la única etapa permitida es MAGI.
Agregaciones	suma, media, mediana y desviacion piden una columna numérica; minimo, maximo y contar sirven con cualquiera.
Faltantes	Una operación con un faltante da faltante y una comparación da falso. DUMMY_PLUG rellena con un valor de la misma clase que la columna.
Conversión	EQUIP falla si algún valor no encaja en el tipo nuevo. Los vacíos siguen vacíos.
Archivos	ENTRY_PLUG falla si el archivo no existe. SEELE solo recibe tablas y solo escribe .csv.
Gráficas	HOLOGRAM solo recibe tablas. Cada tipo exige sus ejes, los ejes tienen que ser columnas de esa tabla y los que se miden (eje_y, o eje_x en el histograma) tienen que ser numéricos. El archivo tiene que terminar en .png o .svg.

La librería de tablas

Está en `src/evangelion/tablas.py` y no depende de nada externo. El intérprete nunca toca los datos directamente: decide qué hay que hacer y se lo pide a esta librería. Tiene tres partes.

Parte	Qué hace
Archivos CSV	Lee el archivo carácter por carácter: respeta las comillas, acepta cualquier separador y decide si cada columna es de enteros, de reales o de texto. Un campo vacío es un faltante. También escribe el CSV de salida.
`Columna`	Una lista de valores con las operaciones fila por fila (aritmética, comparaciones, y, o, no), las estadísticas y las conversiones de tipo.

Parte	Qué hace
`Tabla`	Varias columnas del mismo largo. Sabe seleccionar, filtrar, ordenar por varias columnas, quitar duplicados y faltantes, agrupar, resumir y mostrarse alineada en la terminal.

Algunas decisiones que tocó tomar al escribirla:

Tema	Cómo quedó
Faltantes	Se guardan como None. No cuentan en ninguna estadística.
Media y mediana	La media es la suma entre la cantidad de datos. La mediana es el dato del centro ya ordenados, o el promedio de los dos del centro si la cantidad es par.
Desviación	Es la muestral: la suma de las diferencias al cuadrado se divide entre n - 1 y se saca la raíz. Con un solo dato no se puede calcular y da nulo.
Orden	Se ordena de la última columna a la primera con un ordenamiento estable, y los faltantes van al final.
Grupos	Salen en el orden en que aparecen en la tabla.
Tablas nuevas	Ninguna operación cambia la tabla que recibe; siempre devuelve otra.

Lo único que viene de afuera es el runtime de ANTLR4, que es la herramienta del curso, y pytest, que solo corre las pruebas. Las sugerencias de los errores (*¿Quisiste decir…?*) usan una distancia de Levenshtein escrita a mano.

6. Fase 3: gráficas

`HOLOGRAM` recibe un flujo, igual que `MONITOR` o `SEELE`, y entre llaves las opciones de la gráfica. Las opciones se evalúan dentro de la tabla, así que `eje_x: pais` nombra la columna `pais` y si está mal escrita sale la sugerencia de siempre.

```
HOLOGRAM barras de ultimo {
    eje_x: pais
    eje_y: co2_mt
    titulo: "Emisiones de CO2 en 2023"
    etiqueta_y: "Millones de toneladas de CO2"
    archivo: "caso_estudio/resultados/g1_barras_emisiones_2023.png"
}
```

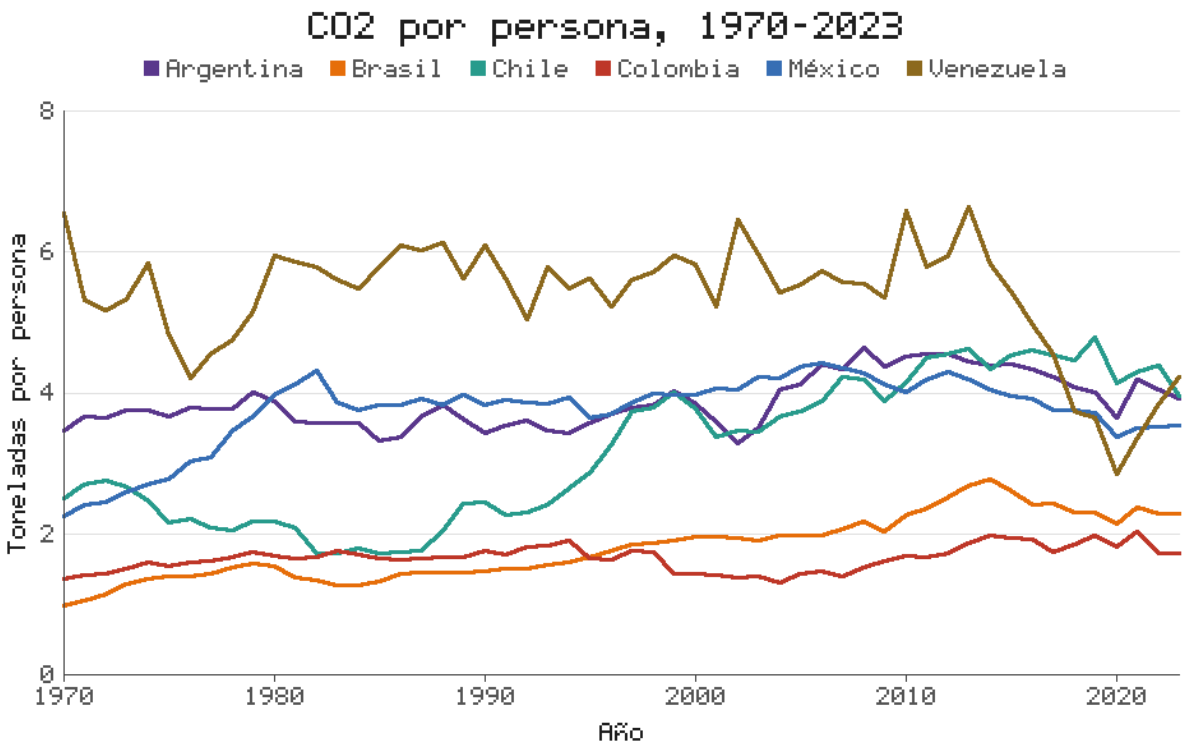
Tipo	Pide	Qué dibuja
`barras`	`eje_x, eje_y`	Una barra por fila con su valor encima. Con serie, barras agrupadas. Si una categoría se repite en la misma serie, sus valores se suman.
`lineas`	`eje_x, eje_y`	Une los puntos en orden de x. Con serie, una línea por grupo.
`histograma`	`eje_x`	Reparte los datos en intervalos del mismo ancho y cuenta cuántos caen en cada uno.
`dispersion`	`eje_x, eje_y`	Un punto por fila; con serie, un color por grupo.

Tipo	Pide	Qué dibuja
`caja`	`eje_y`	Cuartiles, mediana, bigotes hasta 1,5 veces el rango intercuartílico y los datos atípicos como círculos. Con eje_x, una caja por categoría.

Opciones

Opción	Recibe	Para qué
`eje_x`	columna	Lo que va abajo. En barras y caja son categorías; en lineas puede ser número o texto; en histograma y dispersion es numérico.
`eje_y`	columna numérica	Lo que se mide. El histograma no lo usa: cuenta cuántos datos caen en cada intervalo.
`serie`	columna	Separa los datos en grupos de colores con leyenda (barras agrupadas, una línea o una nube de puntos por grupo).
`titulo`	texto	Título arriba de la gráfica.
`etiqueta_x`, `etiqueta_y`	texto	Nombre de cada eje. Si no se dan, se usa el nombre de la columna.
`archivo`	texto	Dónde se guarda. La extensión decide el formato: .png o .svg. Sin esta opción queda en salidas/tipo_n.png.
`color`	texto	Color de la gráfica cuando hay una sola serie: morado, naranja, verde, rojo, azul, amarillo, rosado, gris, negro, cafe o un código #rrggbb.
`intervalos`	entero de 1 a 200	Cuántas barras tiene el histograma. Si no se da, se usa la regla de Sturges.
`ancho, alto`	entero	Tamaño en píxeles. Por defecto 960 x 600.
`leyenda`	verdadero o falso	Muestra u oculta la leyenda de las series.

Las filas a las que les falta algún dato de los ejes se saltan y el mensaje de `HOLOGRAM` dice cuántas se usaron. Las marcas de los ejes son números redondos (pasos de 1, 2, 2,5 o 5 por potencia de 10) y se escriben con coma decimal y punto de miles desde 10.000, para que los años salgan como 1970.



Gráfica de líneas con serie y leyenda (caso de estudio)

Cómo se dibuja por dentro

Está en `src/evangelion/graficas.py` (escalas, ejes, leyenda y los cinco tipos) y en `src/evangelion/lienzo.py` (el dibujo y el formato del archivo). Ninguno usa librerías externas.

Parte	Cómo funciona
Lienzo	Una lista de bytes con un byte por píxel que dice qué color de la paleta lleva. Sabe pintar rectángulos, líneas (algoritmo de Bresenham), círculos y texto.
Letra	Cada carácter es un dibujo de 5 x 7 píxeles hecho a mano en `fuente.py`, con tildes, eñe y signos de apertura. Se agranda multiplicando cada píxel.
Formato PNG	Firma, bloque `IHDR` (tamaño y tipo de color), `PLTE` (la paleta), `IDAT` (los píxeles comprimidos) e `IEND`. Cada bloque lleva su CRC-32, calculado con una tabla propia.
Compresión	DEFLATE con los códigos de Huffman fijos del RFC 1951 y búsqueda de repeticiones LZ77 en dos sitios: el byte anterior (zonas de un color) y el mismo punto de la fila de arriba. Una gráfica de 960 x 600 queda en unos 10-30 KB. Al final va el Adler-32 que pide zlib.
SVG	Un segundo lienzo con los mismos métodos que escribe las figuras como texto. Las gráficas no saben en cuál de los dos dibujan.

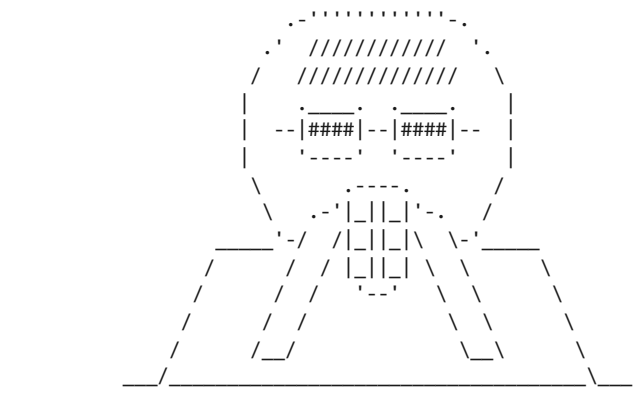
Las pruebas abren cada PNG con el `zlib` de Python para comprobar que el compresor propio produce algo que cualquier programa entiende, y comparan el CRC-32 y el Adler-32 con los de la librería estándar. En el lenguaje no se usa ninguno de los dos.

Errores de las gráficas

```
    Error semántico en sem_08_eje_de_texto.eva, línea 3, columna 34
      tipos incompatibles: en la gráfica de histograma el eje_x tiene que ser numérico y 'piloto' es una
  columna de texto.
        3 | HOLOGRAM histograma de pruebas { eje_x: piloto }
          |                                  ^
```

7. Errores

Hay tres clases de error y todas dicen la línea y la columna, muestran la línea de código y ponen una flecha debajo del sitio exacto. Los mensajes de ANTLR vienen en inglés (*mismatched input*…), así que se traducen antes de mostrarlos. Cuando algo falla aparece el comandante Ikari:



```
    ALERTA: sincronización fallida.
    Nivel de contaminación mental alto, el EVA no arranca.

    Error sintáctico en sin_01_falta_corchete.eva, línea 4, columna 1
      no se esperaba 'MONITOR'. Aquí debería ir ']' o ','.
        4 | MONITOR pilotos
          | ^

    Gendo: "Corrige la línea 4, Shinji. O vete."
```

Un ejemplo de cada clase (sin repetir el dibujo):

Léxico

```
    Error léxico en lex_01_caracter_invalido.eva, línea 3, columna 50
      el carácter '@' no pertenece al lenguaje.
        3 | altas = pruebas |> AT_FIELD donde sincronizacion @ 60
          |                                                  ^
```

Sintáctico

```
    Error sintáctico en sin_02_etapa_inexistente.eva, línea 3, columna 20
      no se esperaba 'FILTRAR'. Aquí debería ir una etapa (TARGET, AT_FIELD, SYNC, SQUAD, MAGI...).
        3 | altas = pruebas |> FILTRAR donde sincronizacion > 60
          |                    ^
```

Semántico

```
    Error semántico en sem_02_columna_inexistente.eva, línea 3, columna 35
      'sincronisacion' no es una columna de la tabla ni una variable. ¿Quisiste decir 'sincronizacion'?
        3 | altas = pruebas |> AT_FIELD donde sincronisacion > 60
          |                                   ^
```

Semántico

```
Error semántico en sem_03_tipos_incompatibles.eva, línea 3, columna 33
  tipos incompatibles: '*' necesita números y recibió una columna de texto.
      3 | raras = pruebas |> SYNC doble = piloto * 2
        |                                 ^
```

El error semántico solo aparece si el programa está bien escrito: primero se revisan todos los errores léxicos y sintácticos, y la ejecución se detiene en el primer error semántico.

8. Programas completos

En `ejemplos/` hay 5 programas que corren de principio a fin y 15 con errores a propósito. Estos dos muestran casi todo el lenguaje.

Resumen por piloto

```
# Limpia las pruebas y saca un resumen por piloto
NERV "Resumen por piloto"

pruebas = ENTRY_PLUG "datos/pruebas_sincronizacion.csv"

# Hay una prueba repetida y algunas sin dato
limpias = pruebas
    |> EJECT duplicados
    |> EJECT faltantes [sincronizacion]
    |> DUMMY_PLUG armonicos = 1.0
    |> SYNC indice = redondear(sincronizacion * armonicos, 1),
            apto = sincronizacion >= 40

por_piloto = limpias
    |> SQUAD por [piloto, unidad]
    |> MAGI pruebas = contar(),
            promedio = media(sincronizacion),
            mejor = maximo(sincronizacion),
            peor = minimo(sincronizacion),
            variacion = desviacion(sincronizacion)
    |> FORMATION por [promedio desc]

MONITOR por_piloto
SEELE por_piloto como "salidas/resumen_pilotos.csv"

# Estadisticas de cada columna
MONITOR limpias |> DIAGNOSTIC
```

```
MONITOR por_piloto (5 filas x 7 columnas)
piloto  unidad  pruebas  promedio  mejor   peor  variacion
Kaworu  EVA-02        1     98.00  98.00  98.00       nulo
Shinji  EVA-01        9     67.18  93.50  55.20      12.39
Asuka   EVA-02        8     66.60  92.70  30.30      20.14
Rei     EVA-00       10     56.84  77.90  43.70       9.88
Toji    EVA-03        1     35.20  35.20  35.20       nulo

SEELE recibió el reporte salidas/resumen_pilotos.csv (5 filas)

MONITOR (9 filas x 9 columnas)
columna         clase     validos  faltantes  distintos  media  desviacion  minimo  maximo
prueba          numero         29          0         29  16.14        9.26       1      31
fecha           texto          29          0         10   nulo        nulo    nulo    nulo
piloto          texto          29          0          5   nulo        nulo    nulo    nulo
unidad          texto          29          0          4   nulo        nulo    nulo    nulo
sincronizacion  numero         29          0         29  63.41       16.45   30.30   98.00
armonicos       numero         29          0         24   1.00        0.22    0.62    1.38
bateria_min     numero         29          0          3   4.28        0.88       3       5
indice          numero         29          0         29  62.34       19.93   33.80  119.60
apto            booleano       29          0          2   nulo        nulo    nulo    nulo
```

El archivo `salidas/resumen_pilotos.csv` queda así:

```
piloto,unidad,pruebas,promedio,mejor,peor,variacion
Kaworu,EVA-02,1,98.0,98.0,98.0,
Shinji,EVA-01,9,67.1778,93.5,55.2,12.3947
Asuka,EVA-02,8,66.6,92.7,30.3,20.1384
Rei,EVA-00,10,56.84,77.9,43.7,9.8836
Toji,EVA-03,1,35.2,35.2,35.2,
```

Informe de combates

```
# El archivo de combates viene con ; y con nombres de columna incomodos
NERV "Informe de combates para el comandante"

combates = ENTRY_PLUG "datos/combates.csv" con {
    separador: ";"
    codificacion: "utf-8"
}
    |> CODENAME [`Angel` -> angel,
                 `Episodio` -> episodio,
                 `Unidad EVA` -> unidad,
                 `Piloto` -> piloto,
                 `Duracion (min)` -> minutos,
                 `Daño (%)` -> dano,
                 `Cable cortado` -> sin_cable]
    |> EQUIP sin_cable -> booleano

# Cuanto daño recibe cada unidad
por_unidad = combates
    |> SQUAD por [unidad]
    |> MAGI combates = contar(),
            dano_medio = media(dano),
            minutos_total = suma(minutos)
    |> FORMATION por [dano_medio desc]

MONITOR por_unidad
SEELE por_unidad como "salidas/dano_por_unidad.csv"

# La bateria interna dura 5 minutos: estos combates fueron los mas apretados
criticos = combates
    |> AT_FIELD donde sin_cable y minutos > 5
    |> SYNC riesgo = redondear(dano * minutos / 100, 2)
    |> TARGET [angel, unidad, piloto, minutos, riesgo]
    |> FORMATION por [riesgo desc]

MONITOR criticos
```

```
dano_general = media(combates.dano)
MONITOR dano_general

PATTERN_BLUE dano_general > 40 {
    SEELE criticos como "salidas/combates_criticos.csv"
} PATTERN_ORANGE {
    MONITOR "Sin novedad, las unidades aguantan"
}

# Si no se dice archivo, la grafica queda en salidas/barras_1.png
HOLOGRAM barras de por_unidad {
    eje_x: unidad
    eje_y: dano_medio
    titulo: "Daño promedio por unidad"
    etiqueta_y: "Daño medio (%)"
    color: "rojo"
}
```

```
MONITOR por_unidad (3 filas x 4 columnas)
unidad  combates  dano_medio  minutos_total
EVA-01        10       52.00          74.60
EVA-00         7       47.43          55.80
EVA-02         8       43.62          69.10

SEELE recibió el reporte salidas/dano_por_unidad.csv (3 filas)

MONITOR criticos (6 filas x 5 columnas)
angel       unidad  piloto  minutos  riesgo
Israfel     EVA-01  Shinji    14.00   10.22
Bardiel     EVA-02  Asuka     13.00    9.49
Bardiel     EVA-00  Rei       10.70    6.42
Sandalphon  EVA-02  Asuka      5.40    4.10
Armisael    EVA-00  Rei        6.70    3.95
Zeruel      EVA-00  Rei       11.20    3.70

MONITOR dano_general: 48.04

SEELE recibió el reporte salidas/combates_criticos.csv (6 filas)

HOLOGRAM proyectó la gráfica de barras en salidas/barras_1.png (3 filas)
```

Los datos de `datos/` son de práctica: los nombres salen de la serie y las cifras son inventadas.

9. Caso de estudio: CO2 en América Latina

El caso final usa datos reales de Our World in Data (licencia CC BY 4.0): emisiones de CO2, población y PIB de 12 países de América Latina entre 1970 y 2023, 648 filas. El documento completo, con la descripción de cada columna, las preguntas, la interpretación y los límites, está en `caso_estudio/README.md`. Aquí va el programa y lo que produce.

```
# Caso de estudio de la fase 3: emisiones de CO2 en 12 paises de America Latina.
# Datos reales de Our World in Data (ver caso_estudio/README.md).
# Se corre desde la carpeta del proyecto:  eva caso_estudio\co2_latam.eva
NERV "Emisiones de CO2 en America Latina, 1970-2023"

# --- 1. Carga y limpieza ------------------------------------------------
co2 = ENTRY_PLUG "datos/co2_latam.csv"
    |> CODENAME [country -> pais, year -> año, population -> poblacion, gdp -> pib,
                co2 -> co2_mt, co2_per_capita -> co2_pc, coal_co2 -> carbon,
                oil_co2 -> petroleo, gas_co2 -> gas, cement_co2 -> cemento,
                energy_per_capita -> energia_pc, share_global_co2 -> parte_mundial]
    |> EJECT columnas [iso_code]
    |> EJECT faltantes [co2_mt, co2_pc]
```

```
    |> EJECT duplicados [pais, año]
    |> SYNC pais = reemplazar(pais, "Brazil", "Brasil"),
            pais = reemplazar(pais, "Mexico", "México"),
            pais = reemplazar(pais, "Peru", "Perú"),
            decada = año - año % 10,
            pib_pc = redondear(pib / poblacion, 0)

MONITOR co2 |> DIAGNOSTIC

# --- 2. Indicadores ----------------------------------------------------------
# P1. ¿Quien emitio mas en el ultimo año con datos?
ultimo = co2
    |> AT_FIELD donde año == 2023
    |> TARGET [pais, co2_mt, co2_pc, parte_mundial]
    |> FORMATION por [co2_mt desc]

MONITOR ultimo
SEELE ultimo como "caso_estudio/resultados/emisiones_2023.csv"

# P2. Promedio y maximo de cada pais en todo el periodo
por_pais = co2
    |> SQUAD por [pais]
    |> MAGI años = contar(),
            co2_pc_medio = media(co2_pc),
            co2_pc_maximo = maximo(co2_pc),
            desviacion_pc = desviacion(co2_pc)
    |> FORMATION por [co2_pc_medio desc]

MONITOR por_pais
SEELE por_pais como "caso_estudio/resultados/resumen_por_pais.csv"

# P3. ¿Colombia esta por encima o por debajo del promedio regional per capita?
colombia = ultimo |> AT_FIELD donde pais == "Colombia"
pc_colombia = media(colombia.co2_pc)
pc_region = redondear(media(ultimo.co2_pc), 3)
MONITOR pc_colombia
MONITOR pc_region

PATTERN_BLUE pc_colombia < pc_region {
    MONITOR "Colombia emite menos CO2 por persona que el promedio de los 12 paises"
} PATTERN_ORANGE {
    MONITOR "Colombia emite mas CO2 por persona que el promedio de los 12 paises"
}

# --- 3. Graficas -------------------------------------------------------------
# G1. Barras: emisiones totales de 2023
HOLOGRAM barras de ultimo {
    eje_x: pais
    eje_y: co2_mt
    titulo: "Emisiones de CO2 en 2023"
    etiqueta_x: "País"
    etiqueta_y: "Millones de toneladas de CO2"
    archivo: "caso_estudio/resultados/g1_barras_emisiones_2023.png"
}

# G2. Lineas: como cambio el CO2 por persona en los seis que mas emiten
grandes = co2
    |> AT_FIELD donde pais == "Brasil" o pais == "México" o pais == "Argentina"
                  o pais == "Venezuela" o pais == "Colombia" o pais == "Chile"

HOLOGRAM lineas de grandes {
    eje_x: año
    eje_y: co2_pc
    serie: pais
    titulo: "CO2 por persona, 1970-2023"
    etiqueta_x: "Año"
    etiqueta_y: "Toneladas por persona"
    archivo: "caso_estudio/resultados/g2_lineas_co2_por_persona.png"
}
```

```
# G3. Histograma: como se reparten los valores de todos los paises y años
HOLOGRAM histograma de co2 {
    eje_x: co2_pc
    intervalos: 16
    titulo: "Distribución del CO2 por persona (12 países, 1970-2023)"
    etiqueta_x: "Toneladas por persona"
    etiqueta_y: "Cantidad de país-año"
    color: "verde"
    archivo: "caso_estudio/resultados/g3_histograma_co2_por_persona.png"
}

# G4. Dispersion: ¿los paises mas ricos emiten mas por persona?
desde_2000 = co2
    |> AT_FIELD donde año >= 2000 y pib != nulo
    |> SYNC pib_pc_miles = pib_pc / 1000

HOLOGRAM dispersion de desde_2000 {
    eje_x: pib_pc_miles
    eje_y: co2_pc
    serie: pais
    titulo: "Riqueza contra emisiones, 2000-2022"
    etiqueta_x: "PIB por persona (miles de dólares de 2011)"
    etiqueta_y: "Toneladas de CO2 por persona"
    archivo: "caso_estudio/resultados/g4_dispersion_pib_vs_co2.png"
}

# G5. Caja: como cambio la diferencia entre paises de una decada a otra
HOLOGRAM caja de co2 {
    eje_x: decada
    eje_y: co2_pc
    titulo: "CO2 por persona por década"
    etiqueta_x: "Década"
    etiqueta_y: "Toneladas por persona"
    color: "morado"
    archivo: "caso_estudio/resultados/g5_caja_por_decada.png"
}

# La misma grafica de lineas tambien sale en SVG, que se ve nitida a cualquier tamaño
HOLOGRAM lineas de grandes {
    eje_x: año
    eje_y: co2_pc
    serie: pais
    titulo: "CO2 por persona, 1970-2023"
    etiqueta_x: "Año"
    etiqueta_y: "Toneladas por persona"
    archivo: "caso_estudio/resultados/g2_lineas_co2_por_persona.svg"
}
```

Lo que imprime (sin el dibujo del EVA-01):

```
MONITOR (14 filas x 9 columnas)
columna        clase   validos  faltantes  distintos            media       desviacion      minimo
maximo
pais           texto       648          0         12             nulo             nulo        nulo
nulo
año            numero      648          0         54          1996.50            15.60        1970
2023
poblacion      numero      648          0        648      34817180.73      46691526.13     1838946
211140731
pib            numero      636         12        636  390536571710.22  596265426527.80  6996917376
3187412907763
co2_mt         numero      648          0        643            92.24           126.58        0.63
556.53
co2_pc         numero      648          0        596             2.32             1.44        0.26
6.63
carbon         numero      503        145        384            10.62            16.04        0.00
69.45
petroleo       numero      648          0        636            57.78            80.15        0.60
```

```
356.75
gas             numero      512        136        476            24.78            34.17        0.00
172.08
cemento         numero      648          0        575             3.54             5.27        0.02
25.80
energia_pc      numero      608         40        608         12217.73          7572.25     1977.54
35695.15
parte_mundial   numero      648          0        352             0.35             0.44        0.00
1.62
... y 2 filas más

MONITOR ultimo (12 filas x 4 columnas)
pais        co2_mt  co2_pc  parte_mundial
Brasil      483.99    2.29           1.27
México      458.25    3.53           1.20
Argentina   178.41    3.92           0.47
Venezuela   119.59    4.23           0.31
Colombia     89.99    1.72           0.24
Chile        77.60    3.95           0.20
Perú         66.06    1.95           0.17
Ecuador      44.60    2.48           0.12
Bolivia      24.62    2.01           0.07
Costa Rica    8.47    1.66           0.02
Paraguay      7.65    1.12           0.02
Uruguay       7.63    2.25           0.02

SEELE recibió el reporte caso_estudio/resultados/emisiones_2023.csv (12 filas)

MONITOR por_pais (12 filas x 5 columnas)
pais        años  co2_pc_medio  co2_pc_maximo  desviacion_pc
Venezuela     54          5.37           6.63           0.81
Argentina     54          3.87           4.64           0.37
México        54          3.74           4.42           0.52
Chile         54          3.19           4.79           1.01
Ecuador       54          1.87           2.73           0.54
Uruguay       54          1.81           2.58           0.37
Brasil        54          1.81           2.78           0.46
Colombia      54          1.66           2.03           0.18
Perú          54          1.29           1.95           0.28
Costa Rica    54          1.28           1.80           0.32
Bolivia       54          1.21           2.01           0.45
Paraguay      54          0.70           1.32           0.28

SEELE recibió el reporte caso_estudio/resultados/resumen_por_pais.csv (12 filas)

MONITOR pc_colombia: 1.72

MONITOR pc_region: 2.592

MONITOR: Colombia emite menos CO2 por persona que el promedio de los 12 paises

HOLOGRAM proyectó la gráfica de barras en caso_estudio/resultados/g1_barras_emisiones_2023.png (12
filas)

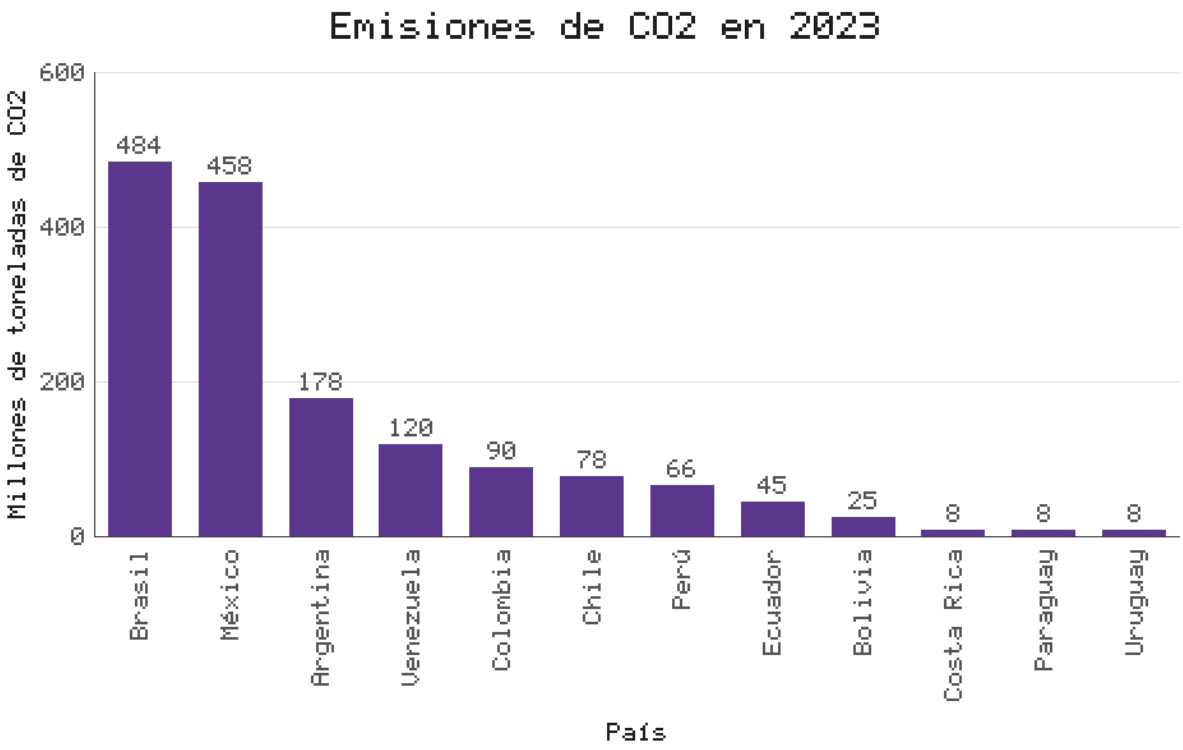
HOLOGRAM proyectó la gráfica de lineas en caso_estudio/resultados/g2_lineas_co2_por_persona.png (324
filas)

HOLOGRAM proyectó la gráfica de histograma en caso_estudio/resultados/g3_histograma_co2_por_persona.png
(648 filas)

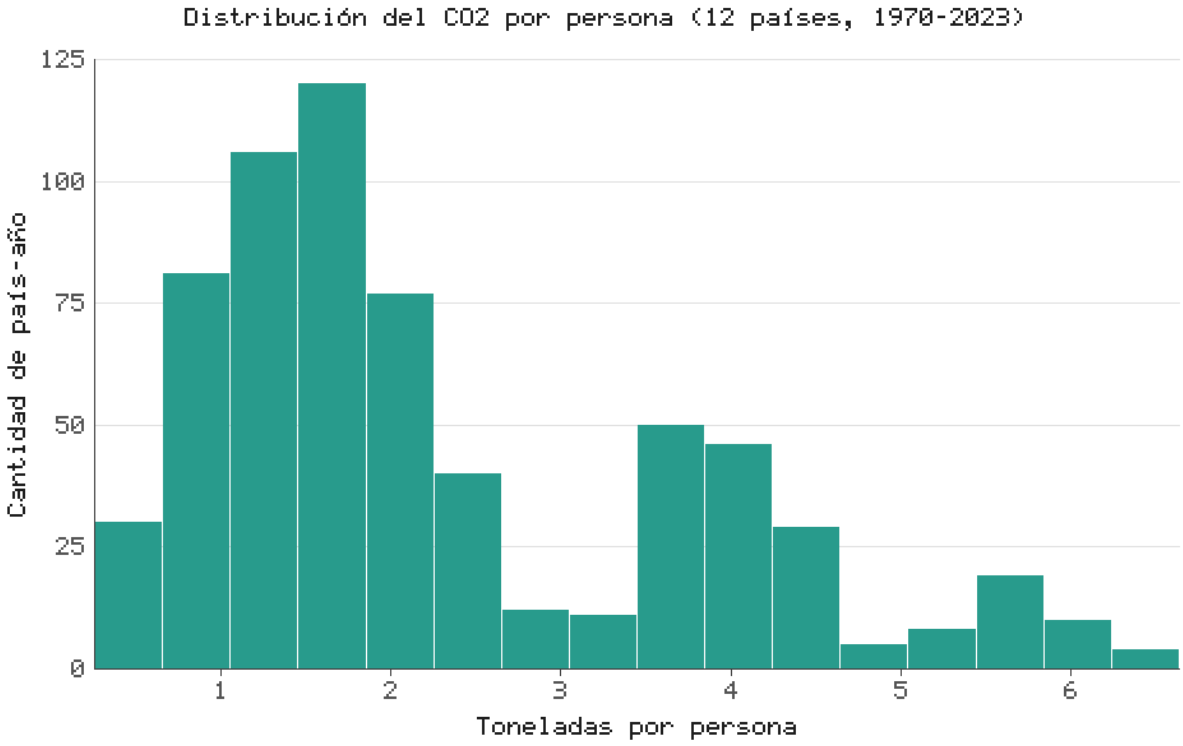
HOLOGRAM proyectó la gráfica de dispersion en caso_estudio/resultados/g4_dispersion_pib_vs_co2.png (276
filas)

HOLOGRAM proyectó la gráfica de caja en caso_estudio/resultados/g5_caja_por_decada.png (648 filas)

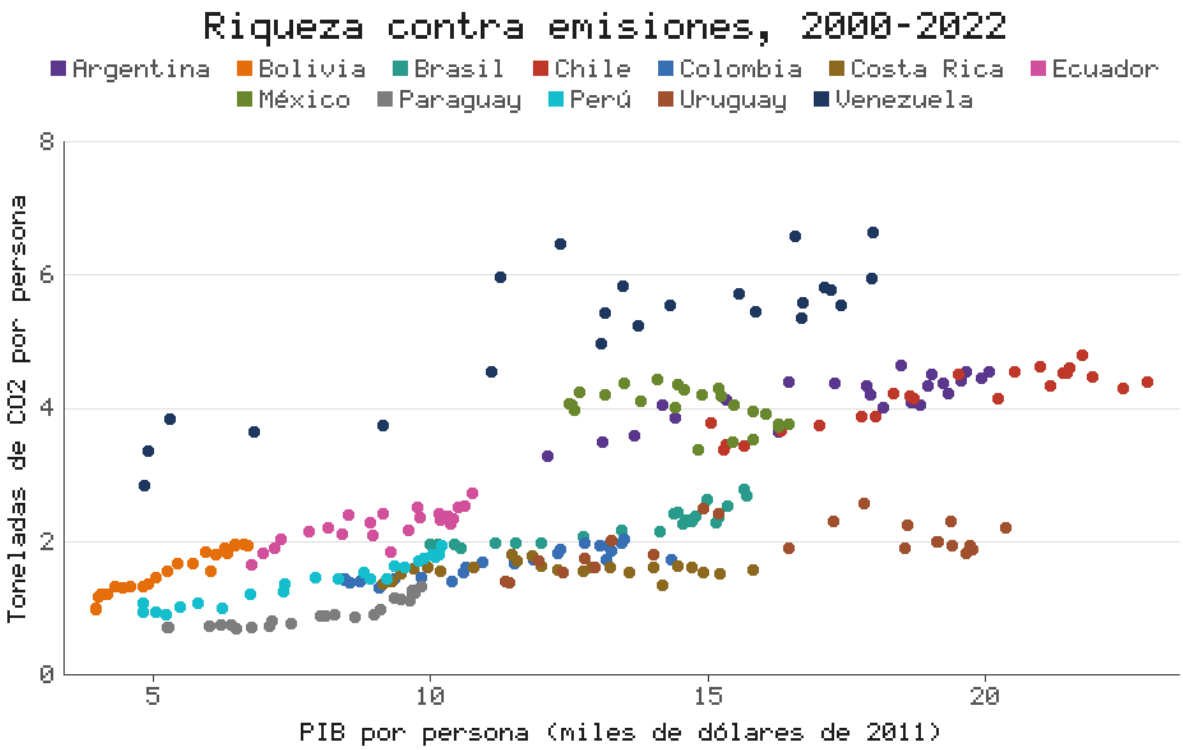
HOLOGRAM proyectó la gráfica de lineas en caso_estudio/resultados/g2_lineas_co2_por_persona.svg (324
filas)
```



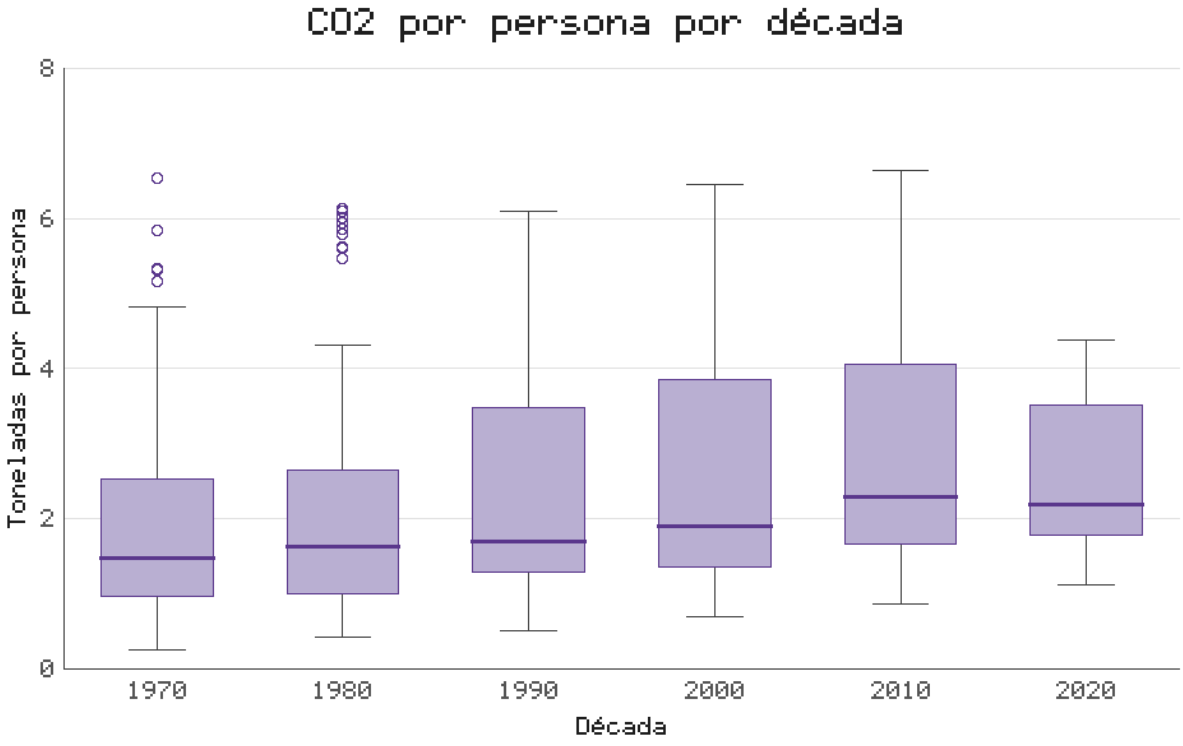
G1. Barras: emisiones totales de 2023



G3. Histograma del CO2 por persona



G4. Dispersión: PIB por persona contra CO2 por persona



G5. Caja: CO2 por persona en cada década

Qué muestran

Brasil y México emitieron en 2023 el 60 % del CO2 de los 12 países, pero por persona los que más emiten son Venezuela, Chile y Argentina. Toda la región está por debajo del promedio mundial (4,71 t por persona). Colombia emite 1,72 t por persona, por debajo del promedio regional de 2,59

t, y es el país con la serie más estable. Hay una relación clara entre PIB por persona y CO2 por persona, con Uruguay y Costa Rica como excepción. La mediana regional subió de 1,48 t en los años 70 a 2,19 t en los 2020.

10. Cómo se usa

```
instalar.bat                                  (crea el entorno, instala y corre las pruebas)
eva ejemplos\01_primera_prueba.eva            (ejecuta un programa)

python run.py programa.eva --tokens           (muestra los tokens)
python run.py programa.eva --arbol            (muestra el árbol en texto)
python run.py programa.eva --svg arbol.svg    (guarda el árbol como dibujo)
python run.py programa.eva --simbolos         (muestra la tabla de símbolos)
python run.py programa.eva --solo-analisis    (fase 1 sin ejecutar)

python -m pytest                              (corre las pruebas)
python regenerar.py                           (vuelve a generar lexer y parser)
```

Las rutas de los CSV se leen desde la carpeta donde se ejecuta el comando, por eso los ejemplos se corren parados en la raíz del proyecto.

Carpeta	Contenido
`grammar/`	`Eva.g4`, la gramática de ANTLR con las reglas léxicas y sintácticas juntas.
`src/evangelion/`	El analizador (fase 1), el intérprete, la tabla de símbolos y la librería de tablas (fase 2), las gráficas, el lienzo y la letra de píxeles (fase 3), los errores, el árbol y los dibujos de la terminal.
`src/evangelion/generado/`	Lo que genera ANTLR. No se edita a mano.
`ejemplos/`	Programas válidos y, en `errores/`, programas que deben fallar.
`datos/`, `salidas/`	Los CSV de entrada y los reportes y gráficas que escriben los ejemplos.
`caso_estudio/`	El caso final: programa, documento, script de descarga y resultados.
`tests/`	Pruebas con pytest de las tres fases.

11. Reporte de pruebas

Las pruebas están en `tests/` y se corren con `python -m pytest`. Las de las fases 1 y 2 se siguen corriendo (solo cambió la que revisaba que `HOLOGRAM` todavía no dibujaba), así que sirven de pruebas de regresión: si la fase 3 hubiera dañado algo de antes, fallarían. Resultado al generar este manual:

Archivo	Qué revisa	Pruebas
`test_fase1.py`	Tokens, precedencia, forma del árbol, errores léxicos y sintácticos con línea y columna.	56
`test_fase2.py`	Cada etapa del flujo, expresiones, tabla de símbolos, errores semánticos y los ejemplos de punta a punta.	89

Archivo	Qué revisa	Pruebas
`test_fase3.py`	Compresor, CRC y PNG contra la librería estándar, la letra, escalas y cuartiles, los cinco tipos en PNG y SVG, las opciones y errores de HOLOGRAM y el caso de estudio completo.	37
`test_tablas.py`	La librería de tablas por separado: CSV, columnas, estadísticas y operaciones de tabla.	26
Total		**208**

```
$ python -m pytest
208 passed in 2.05s
```

Límites conocidos

No hay funciones definidas por el usuario, ciclos ni uniones entre tablas: las etapas ya trabajan sobre toda la tabla y para el caso de estudio no hicieron falta. Tampoco se puede pasar una tabla de formato ancho a largo, por eso una gráfica con varias series necesita una columna que diga a qué serie pertenece cada fila. El PNG usa paleta, que admite hasta 256 colores, y la letra de píxeles no lleva tilde en las mayúsculas.